Лабораторная работа №2

«Метод ломаных»

Выполнил: Рожновский Юрий, K3339, п. 1.2

Преподаватель: Духанов А. В.

## Краткая теоретическая справка

**Метод Пиявского (метод ломаных)** предназначен для поиска глобального экстремума функции на заданном отрезке ([a,b]). Метод основан на построении вспомогательной ломаной, которая ограничивает исходную функцию снизу или сверху.

Для применения метода функция должна удовлетворять **условию Липшица**:

$$|W(u) - W(v)| \leq L|u - v|,$$

где L > 0 – константа Липшица; $u, v \in [a, b]$.

За константу берут максимум производной от функции с дополнительным коэффициентом. Коэффициент 1.2 используется для увеличения точности.

**Построение нижней ломаной.**

Для каждой вычисленной точки (x_i, f(x_i)) строится вспомогательная функция:

g_i(x) = f(x_i) - L|x - x_i|.

Нижняя ломаная определяется как максимум этих функций:

p(x) = max(g_i(x)).

**Вычисление нижних вершин.**

Для двух соседних точек с абсциссами $u_0$ и $u_1$, абсцисса нижней вершины ломаной вычисляется по формуле:

$$u_2 = \frac{W(u_0) - W(u_1)}{2L} + \frac{u_0 + u_1}{2}$$

Её ордината определяется следующим образом:

y = W($u_0$) - L|$u_2$ - $u_0$|.

На каждой итерации среди всех нижних вершин выбирается точка с минимальной ординатой. В найденной точке вычисляется значение исходной функции, после чего нижняя ломаная уточняется.

## Критерий остановки.

В программе используется разность между наименьшим найденным значением функции и минимальным значением нижней ломаной, и итерационный процесс завершается при выполнении условия:

$|W(u_n) - p(u_n)| < \varepsilon$,

где $\varepsilon$ – требуемая точность вычислений.

Дополнительно предусмотрено завершение при достижении максимального количества итераций. Отрицательная разность, превышающая допустимую вычислительную погрешность, рассматривается как признак некорректного построения нижней оценки.

### Объяснение работы программы

Весь код можно найти в файле polygonal_method.py. (https://github.com/hamster1812/5_sem_optimization_methods/blob/main/lab_2/polygonal_method.py)

Код разбит на несколько функций. Разберем их.

```
ALLOALLOWED_MATH = {
    "sin": math.sin,
    "cos": math.cos,
    "tan": math.tan,
    "exp": math.exp,
    "log": math.log,
    "sqrt": math.sqrt,
    "abs": abs,
    "pi": math.pi,
    "e": math.e
}
```

Здесь хранятся имена функций и констант, доступных для использования в выражении.

```
def reading():
    expression = input("Введите функцию: f(x) = ")
    a, b = map(float, input("Введите координаты ограничения a и b через пробел:
").split())
    ext = input("Введите что найти, min или max: ")
    eps = float(input("Введите точность вычисления: "))
    density = input("Введите плотность вычисления ('-' если оставить стандартной
100): ")

    if a >= b:
        raise ValueError("Должно выполняться a < b.")
    if ext not in ("min", "max"):
        raise ValueError("Тип экстремума должен совпадать с min или max.")
    if eps <= 0:
        raise ValueError("Точность eps должна быть положительной.")

    density = 100.0 if density == "-" else float(density)

    return expression, a, b, ext, eps, density
```

Эта функция просто считывает начальные данные для работы и проверяет их на валидность.

```
def take_function(expression, ext="min"):
    count_calls = 0
    def f(x, take_count=False, zero_count=False):
        nonlocal count_calls
        if take_count:
            res = count_calls
            if zero_count:
                count_calls = 0
            return res
        if zero_count:
            count_calls = 0
            return
        count_calls += 1
        return eval(f"-({expression})" if ext == "max" else expression,
{**ALLOALLOWED_MATH, "x": x})
    return f
```

В эту функцию передается выражение функции и тип экстремума, который мы ищем. Возвращает он функцию, которая принимает аргумент x и возвращает его значение для переданного выражения. Также для универсализации последующего решения, если мы ищем максимум, то выражение домножается на -1, чтобы можно было методом ломаных искать только минимум. Реализовано это через лямбда функцию с передачей как аргументов функций, описанных в ALLOALLOWED_MATH. Также с помощью локальных переменных ведется подсчет количества вызовов функции с возможностью обнулить счетчик или взять его значение.

```
def check_values(f, a, b, density):
    num_points = max(2, abs(int((b - a) * density)) * 10)
    x = np.linspace(a, b, num_points)
    try:
        y = np.array([f(xi) for xi in x])
    except (ValueError, ZeroDivisionError):
        raise Exception("Функция определена не на всем рассматриваемом отрезке.")
    except NameError as e:
        raise Exception(f"В функции используются неопределенную математическую
конструкцию: {e.name}")
    if any([not math.isfinite(i) for i in y]):
        raise Exception("Функция содержит точки, не являющиеся конечными.")
```

Эта функция смотрит, что передаваемая функция определена на всем отрезке, не имеет точек со значением бесконечности и выражение функции задано верно.

```python
def find_L(f, a, b, density):
    num_points = max(2, abs(int((b - a) * density * 10)))

    x = np.linspace(a, b, num_points)
    y = np.array([f(xi) for xi in x])
    dx = np.diff(x)
    dy = np.abs(np.diff(y))
    L = np.max(dy / dx)

    return L * 1.2
```

Эта функция используется для нахождения константы Липшица (L). Она разбивает отрезок на множество отрезков, количество которых зависит от длины изначального отрезка и выбранной плотности. После этого вычисляются значения на границах подотрезков и берется максимальное значения наклона прямой, проходящей через соседние точки. Дополнительно полученное значение домножается на 1.2 для избегания погрешностей.

```python
def find_crossing_points(points, L, ext="min"):
    crossing_points = []

    for (x1, y1), (x2, y2) in zip(points, points[1:]):
        if L == 0:
            crossing_points.append(((x1 + x2) / 2, y1))
            continue
        x_cross = (x1 + x2) / 2 + (y1 - y2) / (2 * L)
        if x_cross <= x1 or x_cross >= x2:
            raise Exception(
                f"Выбран слишком низкий коэффициент Липшица ({L}), "
                f"ломанная пересекает график функции на подотрезке ({x1}, {x2})."
            )
        y_cross = y1 + (1 if ext == "max" else -1) * L * abs(x_cross - x1)
        crossing_points.append((x_cross, y_cross))

    return crossing_points
```

Эта функция принимает точки на графике функции, константу L и при необходимости тип вычисляемого экстремума. Дальше мы разбиваем их на соседствующие пары и для каждой пары по формуле находим x для точки пересечения ломаной и по формуле находим y точки пересечения. Если x выходит за границы подотрезка, то вызывается исключение, говорящее о том, что выбран слишком низкий коэффициент L. Добавляем найденную точку в массив и потом возвращаем весь массив.

```python
def piyavskii(f, a, b, L, eps, max_iter=None):
    if max_iter is None:
        max_iter = max(1, b - a) * 1500
    start_time = time.perf_counter()
    iterations = 0
    delta = math.inf
    flag = "max_iter"

    points = [(a, f(a)), (b, f(b))]

    while iterations < max_iter:
        points.sort()
        best_y = min(y for x, y in points)
        low_points = find_crossing_points(points, L, ext="min")

        x_new, lower_min = min(low_points, key=lambda p: p[1])

        delta = best_y - lower_min
        if delta < -1e-10:
            flag = "neg_delta"
            break
        if delta < eps:
            flag = "normal"
            break

        y_new = f(x_new)
        if not math.isfinite(y_new):
            raise Exception("Функция содержит точки, не являющиеся конечными.")
        points.append((x_new, y_new))
        iterations += 1

    elapsed_time = time.perf_counter() - start_time

    return points, iterations, elapsed_time, flag, delta
```

Это функция выполнения непосредственно метода Пиявского. Для статистики мы замеряем время работы и количество выполненных операций. Основной принцип работы: мы имеем массив вычисляемых точек данной функции. На каждой итерации среди нижних точек ломаной находим точку с наименьшим значением и по полученному x добавляем новую точку в массив точек графика функции. Причиной остановки выступают 3 ситуации: когда разница наименьших значений функции и ломаной стала отрицательной; когда эта разница стала меньше требуемой точности; когда было превышено допустимое количество операций. Возвращается все полученные

точки функции, кол-во итераций, время работы метода, тип остановки и разницу.

```
def calculation(expression, a, b, ext, eps, density):
    f = take_function(expression, ext)
    check_values(f, a, b, density)
    L = find_L(f, a, b, density)
    f(0, zero_count=True)

    if L == 0:
        print("Константа L = 0, возможно взята маленькая плотность или функция
постоянна.")
        f_calls = f(0, take_count=True)
        if ext == "max":
            f = take_function(expression)
        print_stats(L, (a, f(a)), 0, f_calls, 0, "zero_L", 0)
        visualize(f, expression, a, b, ext, L, density, ((a, f(a)), (b, f(b))),
(a, f(a)))
        return

    points, iterations, elapsed_time, flag, delta = piyavskii(f, a, b, L, eps)

    best_point = min(points, key=lambda p: p[1])

    f_calls = f(0, take_count=True)

    if ext == "max":
        points = [(x, -y) for x, y in points]
        best_point = (best_point[0], -best_point[1])
        f = take_function(expression)

    print_stats(L, best_point, iterations, f_calls, elapsed_time, flag, delta)
    visualize(f, expression, a, b, ext, L, density, points, best_point)
```

Это функция, объединяющая все выполнение. По заданным данным находятся функция (под поиск минимума) и коэффициент L, применяется метод Пиявского, если мы изначально перевернули функцию, переворачиваем снова, в том числе найденные точки, после чего выводим статистику и визуализируем графики. Отдельно обрабатывается случай, когда L = 0.

Код для статистики и визуализации разбирать не вижу особого смысла в рамках задания лабораторной.

## Примеры работы

### Пример 1

Пусть начальные данные следующие:

```
expression = "sin(x)"
a, b = 0, 13
ext = "min"
eps = 0.1
density = 100
```

Получим статистику:

```
Результаты работы метода Пиявского
-----------------------------------
Коэффициент Липшица:        L = 1.200000
Количество вызовов функции: 21
Причина завершения:         достигнута погрешность (0.09786282619961473)
Точка экстремума:           x = 4.716163
Значение функции:           f(x) = -0.999993
Количество итераций:        19
Время работы:               0.000375 сек.
```

Рисунок 1 - Статистика метода для примера 1

И график:

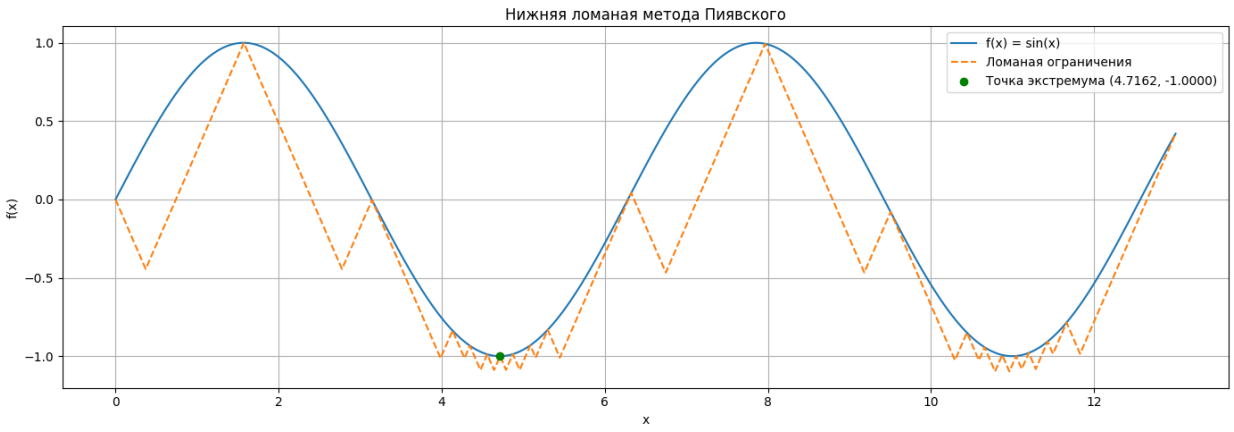



Рисунок 2 - График функции и ломаная примера 1

### Пример 2

Рассмотрим функцию Растригина со следующими параметрами:

```
expression = "10 + x**2 - 10 * cos(2 * pi * x)"
a, b = -1.5, 2
ext = "max"
```

```
    eps = 0.01
    density = 100
```

Статистика:

```
Результаты работы метода Пиявского
----------------------------------
Коэффициент Липшица:        L = 78.398855
Количество вызовов функции: 177
Причина завершения:         достигнута погрешность (0.009933806024509 55)
Точка экстремума:           x = 1.507608
Значение функции:           f(x) = 22.261459
Количество итераций:        175
Время работы:               0.011569 сек.
```

Рисунок 3 - Статистика метода для примера 2

И график:

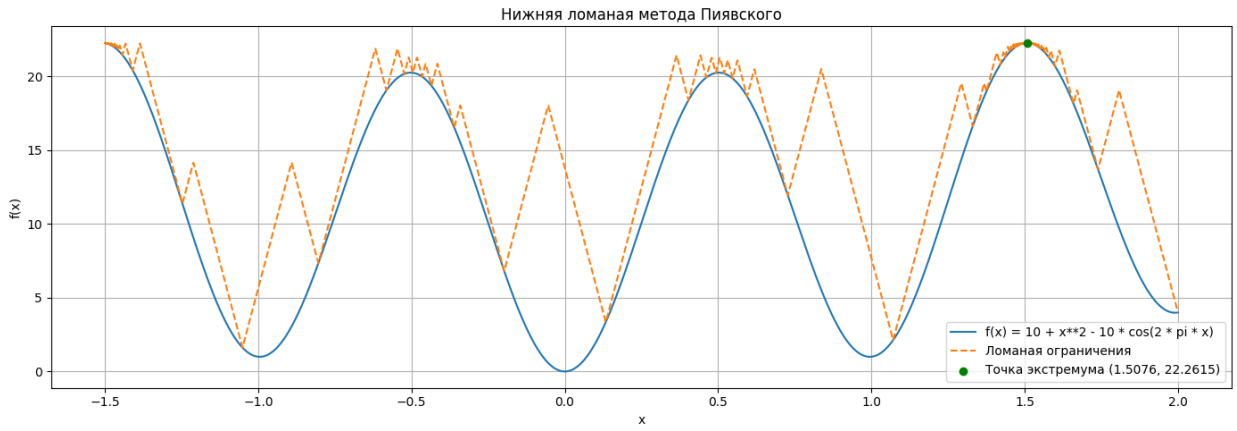



Рисунок 4 - График функции и ломаная примера 2

Пример 3

Рассмотрим функцию Растригина с теми же параметрами, но найдем минимум:

```
    expression = "10 + x**2 - 10 * cos(2 * pi * x)"
    a, b = -1.5, 2
    ext = "min"
    eps = 0.01
    density = 100
```

Получим статистику:

```
Результаты работы метода Пиявского
----------------------------------
Коэффициент Липшица:        L = 78.398855
Количество вызовов функции: 155
Причина завершения:         достигнута погрешность (0.009765742302682678)
Точка экстремума:           x = 0.000057
Значение функции:           f(x) = 0.000001
Количество итераций:        153
Время работы:               0.008787 сек.
```

Рисунок 5 - Статистика метода для примера 3

И график:

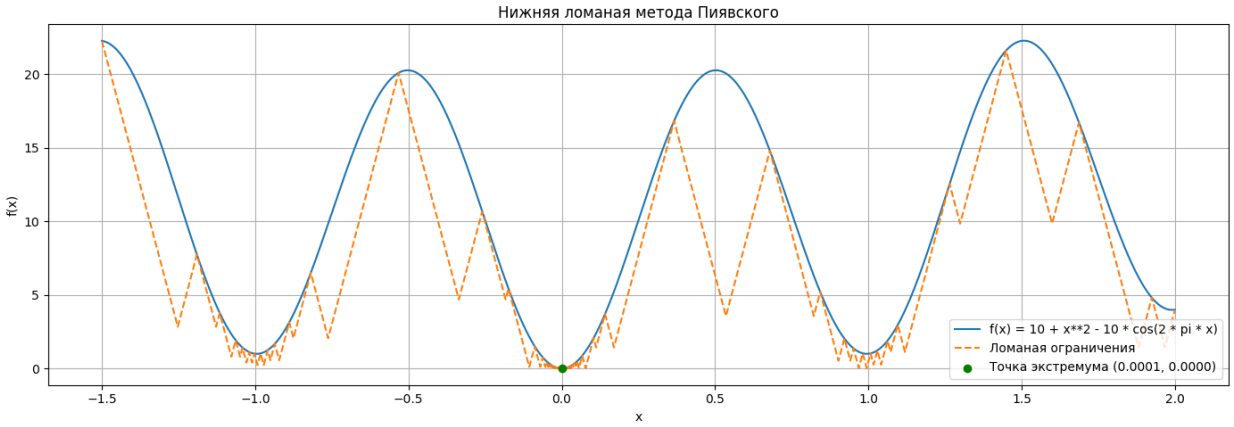



Рисунок 6 - График функции и ломаная примера 3

## Вывод

Был изучен способ поиска экстремумов на отрезке функции при помощи метода Пиявского. Также была написана программа для применения метода автоматически.